

PRELIMINARY DESIGN REPORT (PDR)

NIOT STUDENT AUTONOMOUS UNDERWATER VEHICLE (AUV) COMPETITION

Table of Contents

Section	Content	Subtopics
1	Team Information	1.1 Team Roles
2	Analysis of Mission	2.1 Mission overview 2.2 Mission Specifics 2.3 Mission Timeline 2.4 Design Requirements
3	System Overview	3.1 Summary of our Vessel
4	Mechanical Design	4.1 Vehicle Architecture 4.2 Frame Design 4.3 Pressure Hull Configuration 4.4 Payload Integration
5	Propulsion System	5.1 Overview and Geometry 5.2 Vehicle Maneuverability and Electronic Speed Controllers (ESC)
6	Electronics System	6.1 Overview and Processing Layers 6.2 Components List 6.3 Component Placement 6.4 Connection Analysis 6.5 Connection Diagram
7	Power System	7.1 Power Requirements 7.2 Main Battery Selection 7.3 Battery Specifications 7.4 Power Distribution Architecture 7.5 Battery Management and Balancing Safety
8	Software & Simulation Architecture	8.1 Overview 8.2 Architectural Details 8.3 Simulation Environment (SITL) 8.4 Digital Twin Vehicle Profile
9	Navigation Stack	9.1 Sensor Fusion and Output Data Packets

		9.2 Frame Transformation Matrix 9.3 Extended Kalman Filter based Localization 9.4 Path Planning and Waypoint Generation
10	Control System Design & Tuning	10.1 Cascaded PID Architecture 10.2 6-Thruster Allocation Matrix 10.3 Nonlinear PWM Conversion 10.4 Controller Loop Tuning Parameters
11	Communication Systems	11.1 System Interface Layout 11.2 Kill Switch Compliance
12	Safety Analysis	12.1 Safety Features 12.2 Hazard Analysis (FMEA)
13	Simulation and Testing	13.1 Simulation Test Results 13.2 Hardware-in-the-Loop Results 13.3 In-Water Testing Timeline
14	Competition Timeline	

1. Team Information

Team Name: Nautilus

Institution: Indian Institute of Technology Kharagpur

Department: Ocean Engineering and Naval Architecture

Faculty Advisor: Prof. Jagadeesh Kadiyam

Team Leader: <Team Leader>

Team Members: Adyasha Patnaik, Sahith Reddy Reddivari, Arpit Aggarwal, Aditya Gupta

Contact Email:<insert>

1.1 Team Roles

Name	Role	Subsystem Responsibility
<Team Leader>	Team Lead	Design, Fabrication, Communications
Adyasha Patnaik	Team Member	Electronics and Power

		System, Navigation
Sahith Reddy Reddivari	Team Member	Design and mechanical
Arpit Aggarwal	Team Member	Navigation and Communications
Aditya Gupta	Team Member	Power Systems and Controls

2. Analysis of Mission

2.1. Mission Overview

The primary objective of the Autonomous Underwater Vehicle (AUV) is to execute an entirely autonomous underwater run through a structured series of visual, spatial, and mechanical tasks without human intervention or global positioning markers (GPS).

2.2. Mission Specifics

Available Tank Specifications for the task:

Length: 25 Meter

Breath: 20 Meter

Water height: 2 to 2.5 Meters

2.3. Mission Timeline

1. Launch: Start from the designated NIOT launching point.
2. Descent & Navigation: Dive down and track the orange path.
3. Task 1 (Collect Flowers): Touch at least one buoy.
4. Task 2 (Gate Navigation): Follow the orange path and pass over the L-shaped PVC bar.
5. Route Choice: Select a path at the fork, with the option to use the connecting path to complete both if time permits:
 - Left Route (Drop Markers): Drop markers (plastic/metal sheets) into the bins.
 - Right Route (Shoot Torpedo): Fire at least one torpedo through the heart-shaped cutout.
6. Recovery: Follow the remaining path and surface fully within the octagon.

2.4. Design Requirements

After analysis of the mission, following are the calculated assumptions that were made as to the minimum requirements to complete the full course within the time limit:

1. Kinematic Velocity and Metric Sizing Assumptions

- Maximum Target Station-Keeping Drift: During close-range operations (Bin Drop, Torpedo Alignment), the combined EKF loop is assumed to bound horizontal drift to less than $\pm 0.05\text{m}$.
- Waypoint Acceptance Radius: The cross-track trajectory controller assumes an acceptance sphere radius of 0.2m . Setting the threshold any tighter causes the vehicle to dramatically decelerate while a larger radius risks missing the physical triggers for visual tracking.

2. Optical Flow and Altitude Sensing Constraints

- Minimum Operating Altitude: The navigation stack assumes the AUV maintains a clear distance of at least 0.5m above the pool floor. If the vehicle drops below this limit, the PMW3901 sensor's lens field of view shrinks causing motion blur
- Pool Floor Feature Density: The system operates on the calculated assumption that the pool bottom contains at least 10% distinct visual contrast so that the optical sensor can compute correct frame-to-frame pixel shifts

3. Computational Latency & Power Budgets

- Maximum End-to-End Control Latency: The system assumes a maximum latency of 30ms from the moment the PMW3901 registers a velocity change till the Navigator updates the PWM pulse to the ESCs.
- Battery Operating Life: The 1200mAh parallel 4S LiPo battery array is calculated to sustain a total continuous mission current draw of 35A under typical multi-axis loads. This provides an absolute maximum operational window of 20.5 mins, ensuring the completion of the mission.

3. System Overview

3.1. Summary of our Vessel

1. Hull Form: Open Frame with Dual Watertight Cylinders
2. Length Overall (LOA) : 0.46m
3. Full Breadth (B) : 0.33m
4. Design Draft (T) : Fully Submersible
5. Hull Material : Clear cast acrylic (PMMA)

6. Propulsion System : 6 x BlueRobotics T100 Thrusters
7. Battery : 4S LiPo, 14.8V, 12000mAh
8. Autopilot : ArduSub Firmware (Configured for Vectored/Differential Subsea Flight)
9. Main Computer : Raspberry Pi 4B
10. Flight Controller : Navigator
11. Sensor Suite : Intel RealSense D435i, CSI Downward Camera, Bar30 Depth Sensor, PMW3901 Optical Flow Breakout, XSENS IMU, Current/Voltage Sensors
12. Communication : MAVLink over UDP

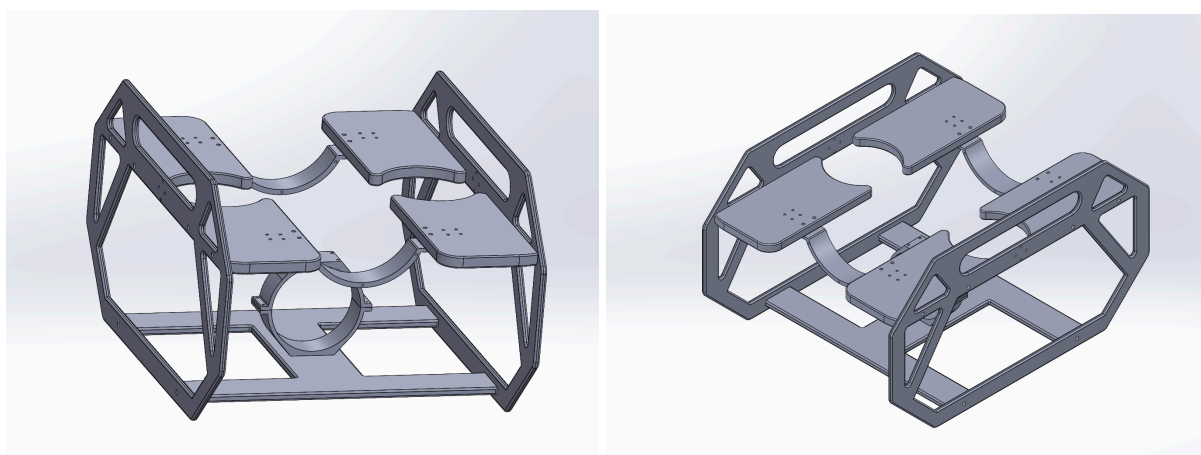
4. Mechanical Design

4.1. Vehicle Architecture

The mechanical design of the AUV was developed with emphasis on modularity, structural rigidity, and ease of maintenance. Since the vehicle will operate in a dynamic underwater environment, the frame was designed to provide a rigid platform while allowing sufficient space for the pressure hulls, propulsion system, and payload mechanisms.

An open-frame architecture was selected to simplify manufacturing and assembly while also reducing overall weight. The design allows individual subsystems to be removed or replaced without requiring complete disassembly of the vehicle, making maintenance considerably easier during testing and competition.

The final frame consists of two identical side plates connected using longitudinal structural members and dedicated pressure hull supports, forming a compact yet rigid chassis.

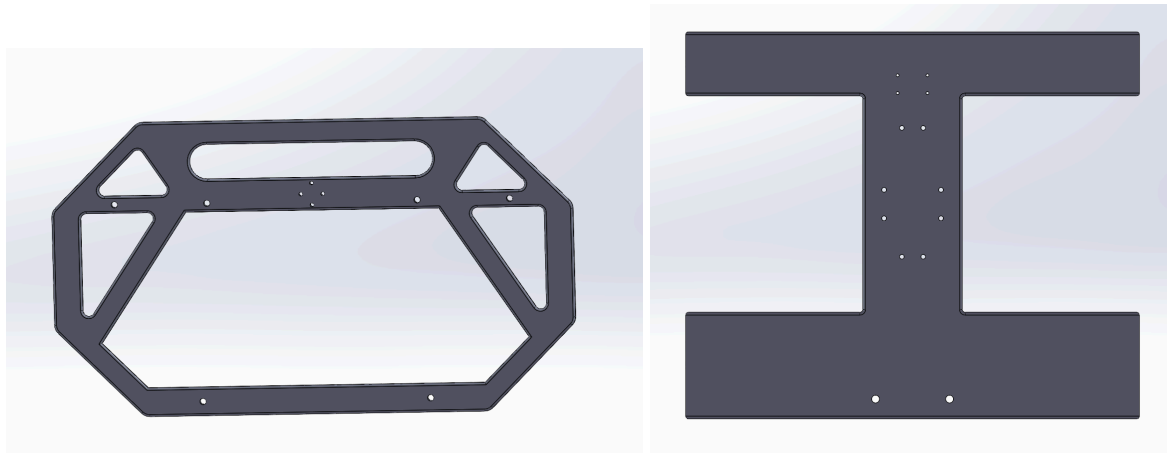


4.2. Frame Design

The frame consists of two identical side plates connected by longitudinal members, forming the primary structure of the AUV. It provides mounting interfaces for the pressure hulls,

thrusters, and payload mechanisms while maintaining an open design for easy assembly and maintenance. Weight-reducing cut-outs and rounded corners improve manufacturability, reduce mass, and provide convenient routing for wiring. The frame is manufactured from Aluminium 6061-T6 due to its high strength-to-weight ratio, corrosion resistance, and ease of machining.

The bottom plate acts as the primary mounting platform for the torpedo launcher, marker dropper, and downward-facing Raspberry Pi Camera Module 3 Wide. Its modular mounting pattern allows payloads to be easily installed or replaced while increasing structural rigidity and maintaining a low centre of gravity for improved underwater stability.



4.3. Pressure Hull

To improve modularity and simplify the internal layout, the AUV uses two independent pressure hulls instead of a single enclosure. The upper pressure hull houses the primary electronics, including the Raspberry Pi 4B, Blue Robotics Navigator, XSENS IMU, power distribution board (PDB), 5V/6A BEC, ESCs, and the forward-facing Intel RealSense camera.

The lower pressure hull is dedicated to the battery system, housing the dual 4S LiPo batteries. Separating the batteries from the electronics improves maintenance, reduces thermal effects on sensitive components, and allows the batteries to be replaced without disturbing the main electronics.

Both pressure hulls are constructed using cast acrylic tubes with Aluminium 6061-T6 end caps sealed using Nitrile (NBR) O-rings. This design provides a reliable waterproof enclosure while allowing easy access for maintenance and subsystem integration.



Upper pressure vessel



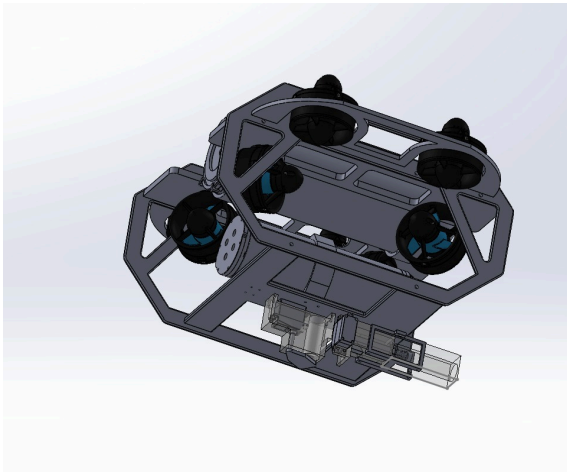
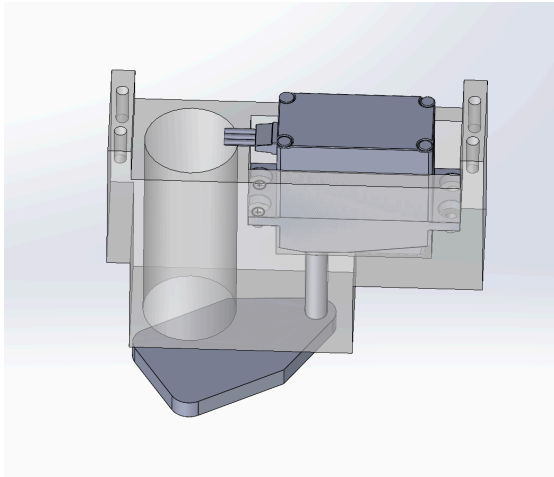
Lower pressure vessel

4.4. Payload Integration

Two mission-specific payload mechanisms are incorporated into the baseline vehicle.

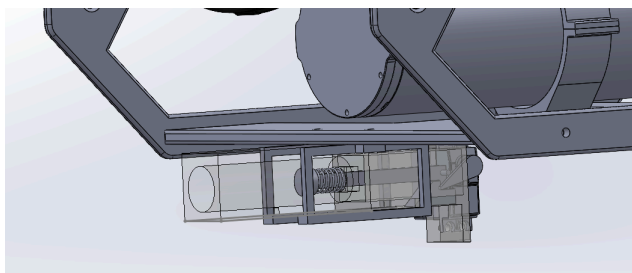
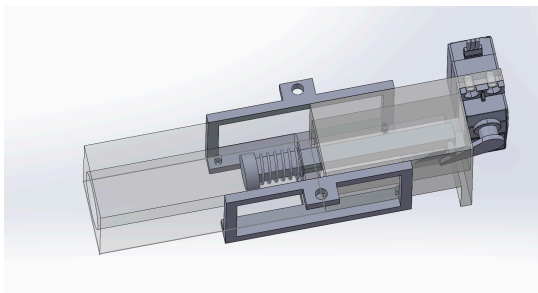
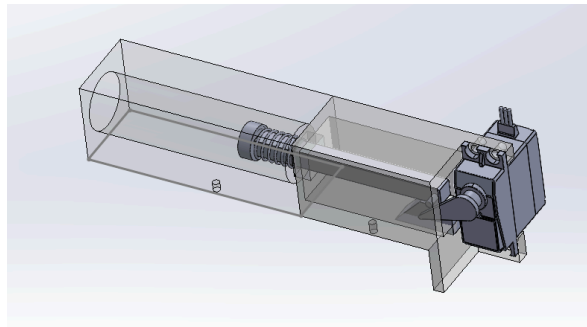
Marker Dropper

The marker dropper is designed to release markers individually using a compact servo-actuated rotary mechanism. The markers are stored vertically inside a cylindrical holder, while a waterproof servo rotates a shaft connected to the release mechanism. During normal operation, the mechanism supports the markers and prevents them from falling. When a deployment command is received, the servo rotates the shaft, creating an opening that allows a single marker to drop under gravity. After the marker is released, the servo returns to its original position, securing the remaining markers for subsequent deployments.



Torpedo Launcher

The torpedo launcher employs a spring-loaded linear mechanism enclosed within a protective housing. A compression spring stores the required energy while a sliding piston, guided by a central rod, ensures smooth and aligned linear motion during operation. The spring is held in its compressed state by a servo-actuated latch. Upon receiving a deployment command, the servo disengages the latch, allowing the spring to expand and drive the piston forward, ejecting the torpedo. The enclosed design protects the internal components, improves reliability, and allows the launcher to be integrated as a compact modular subsystem within the AUV.

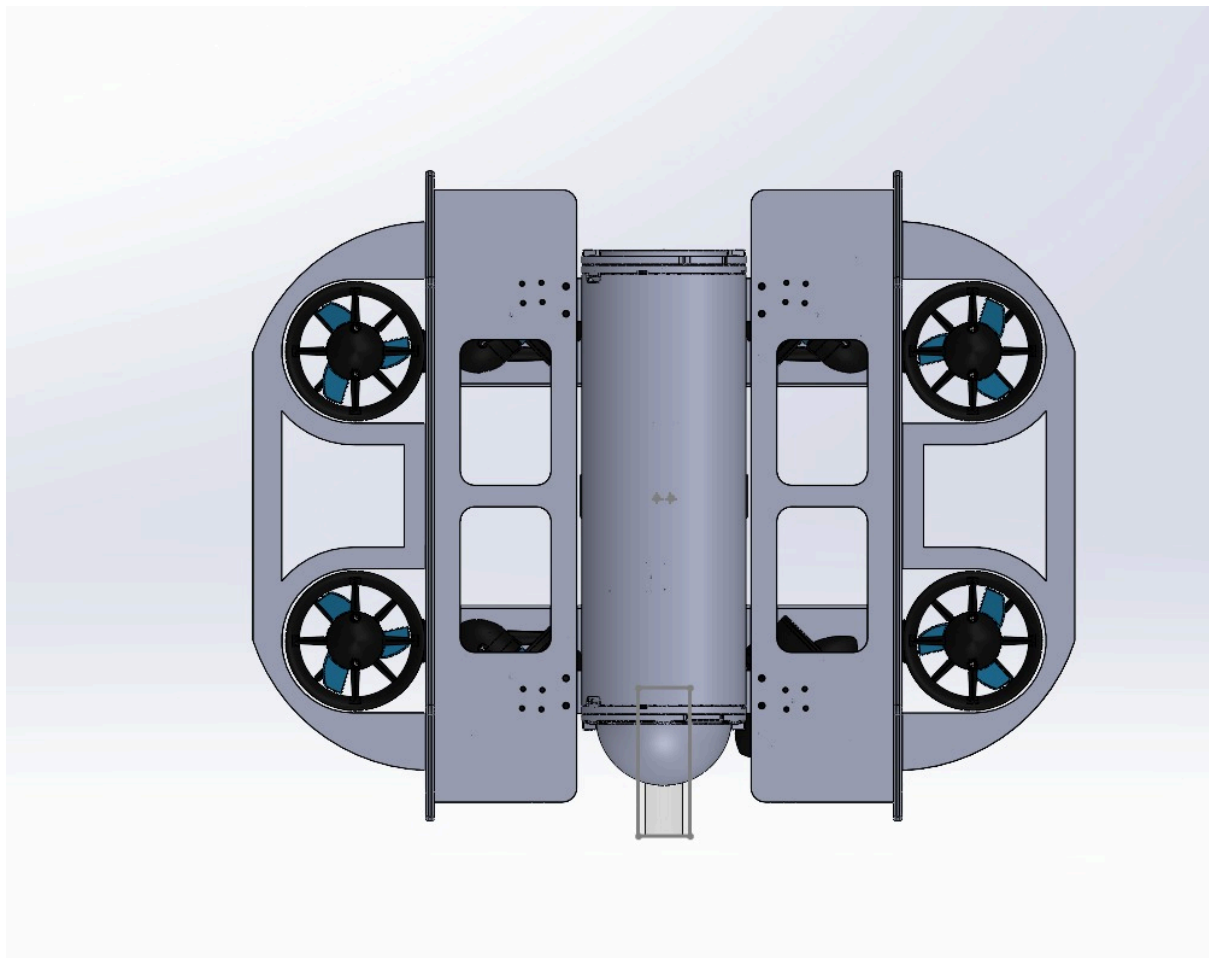


5. Propulsion System

5.1. Overview

The AUV employs a six-thruster vectored propulsion system to provide full six-degree-of-freedom (6-DOF) maneuverability. The configuration consists of four horizontal thrusters mounted at 45° to generate surge, sway, and yaw motions, while two vertically mounted thrusters provide heave control and assist in maintaining pitch and roll stability.

The thrusters are symmetrically positioned on the frame to ensure balanced thrust distribution, minimize unwanted moments, and improve overall vehicle stability. This configuration enables precise maneuvering in confined underwater environments while maintaining efficient propulsion during mission execution. Also each thruster mounted using dedicated brackets to allow easy maintenance and replacement.



5.2. Vehicle Maneuverability and Electronic Speed Controllers (ESC)

The AUV uses a six-thruster vectored propulsion system to achieve full 6 DOF maneuverability. Four thrusters are mounted horizontally at 45° to provide surge, sway, and yaw motion, while two vertically mounted thrusters control depth and assist in maintaining pitch and roll stability. This configuration enables the vehicle to perform precise movements, station keeping, and accurate navigation in confined underwater environments.

Each thruster is driven by an independent ESC, which receives PWM control signals from the flight controller and regulates the speed and direction of the brushless motors. Using a dedicated ESC for every thruster provides precise thrust control, improves system reliability, and allows smooth coordinated motion during autonomous operation. The ESCs are housed inside the electronics pressure hull, protecting them from the underwater environment while keeping the electrical system compact and organized.

6. Electronics System

6.1. Overview

The electronics architecture of the AUV is built on a modular design that separates high-bandwidth processing from real-time execution, while keeping high-current paths isolated from sensitive logic lines. This is to manage computational load and prevent noise interference across subsystems.

6.2. Components List

Component	Model	Function
Main Computer	Raspberry Pi 4B	ROS2 based Sensor Fusion, EKF, Mission Planning, PID/MPC Control, logging
Flight Controller	Blue Robotics Navigator	Intercepts subsea telemetry, passes sensor frames to the Pi, and outputs hardware PWM motor commands.
IMU	XSENS 630RMti	6-DOF inertial data Magnetometer data
Forward facing Camera	Intel RealSense D435i	Captures front-view data

		for target alignment, gate tracking, and object visual servoing
Downward Camera	Raspberry Pi Camera Module 3 Wide	Streams real-time frames of the pool floor for orange path
ESC	6x Blue Robotics Basic ESCs	Power interface from battery to propellers
Battery	2x 4S LiPo	Main power supply
Pressure sensor	Bar30 Sensor (MS5837 chip)	Vertical underwater depth measurement
Optical flow sensor	PMW3901 breakout board	Tracks pixel shifts locally to output absolute planar velocities and bound IMU drift.
Marker Dropper Servo	DSServo DS3225	Rotates a small trapdoor or pulls a pin to let the marker drop under gravity
Torpedo Firing Servo	DSServo DS3235	Disengages a mechanical latch to release a compressed spring, firing the torpedo

6.3 Placement of Components

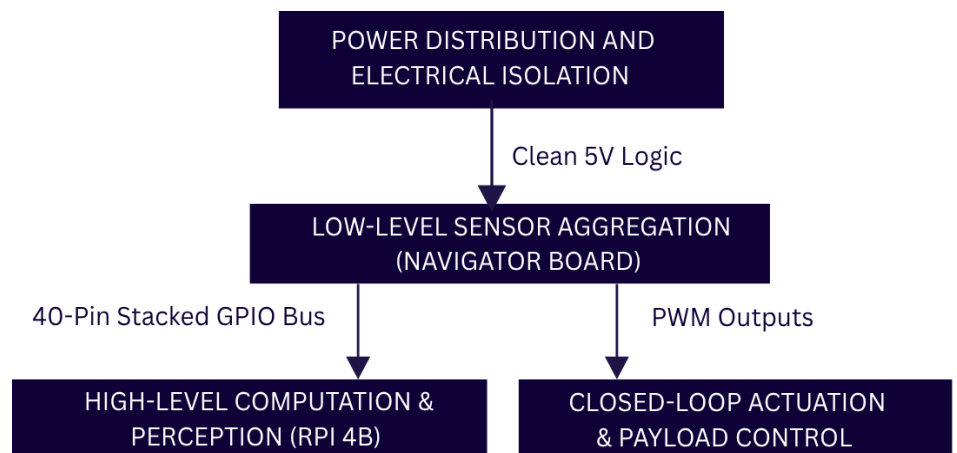
We have separated our AUV into two distinct watertight enclosures for battery and other electronics components. The LiPo batteries are kept in a different chamber than processing units because they require additional heat management.

Component	Position
Main Computer	Electronics Enclosure
Flight Controller	Electronics Enclosure
IMU	Electronics Enclosure
Pressure sensor	Electronics Enclosure - Rear End Cap
Forward facing camera	Electronics Enclosure - Front Exterior

Downward facing camera	Electronics Enclosure - Bottom Exterior
Optical flow breakout board	Electronics Enclosure - Bottom Window (Air-Gap)
ESC	Vessel compartment (Isolated Tray)
Battery	Battery Enclosure

6.4 Connection Analysis

There are four divisions which form a decoupled engineering architecture designed for our autonomous underwater vehicle. Splitting the system this way addresses the three biggest failure points in marine robotics: electrical noise, data processing bottlenecks, and real-time control latency.



1. Power Distribution and Electrical Isolation

The electrical backbone originates from 2x 4S LiPo batteries linked via high-current XT60 connectors to a central Power Distribution Board (PDB). The power delivery splits into two distinct paths to separate high-noise propulsion elements from sensitive digital logic:

Propulsion Rail: The PDB delivers a raw 4S voltage line directly to the Brushless Electronic Speed Controllers (ESCs). The ESCs convert this DC power into 3-phase AC power to drive the Brushless Thrusters.

Logic Rail: A raw 4S line feeds into a 5V / 6A Battery Eliminator Circuit (BEC), acting as a dedicated power module. This steps down the variable battery voltage into a clean, regulated 5V rail that powers the Navigator Flight Board, which subsequently shares it with the Raspberry Pi 4B via a direct physical hardware interface. This configuration isolates the core processors from voltage spikes and electromagnetic interference (EMI) generated by the thrusters.

2. High-Level Processing and Perception (Raspberry Pi 4B)

The Raspberry Pi 4B acts as the primary computational hub, running ROS2 environment, the top-level mission state machine, and spatial Extended Kalman Filter (EKF) localization algorithms. By keeping the processing on the Pi, the low-level flight controller is never bogged down by CPU-intensive software routines. This unit directly manages the high-bandwidth perception sensors as follows:

Inertial Measurement Unit (IMU): Connected via a USB 2.0 link to provide high-frequency raw angular velocity and linear acceleration payloads.

Forward Camera: An Intel RealSense D435i depth camera interfaces via a high-speed USB 3.0 High port to stream uncompressed, real-time spatial video frames for forward obstacle detection and target tracking.

Downward Camera: A path-following camera interfaces directly via a 15-Pin Flat CSI Ribbon cable to pipe video frames straight to the GPU, minimizing CPU overhead during visual line-following tasks.

3. Low-Level Sensor Aggregation (Navigator Flight Board)

The Navigator Flight Board sits underneath the Pi via an Internal GPIO Stacking Connection. It serves as a real-time sensor aggregator and hardware interface layer:

Bar 30 Depth & Temperature Sensor: Communicates over an I2C Bus, sending real-time ambient pressure and water temperature data to the Navigator.

Optical Flow Sensor: Features an onboard microcontroller that tracks physical pixel shifts along the pool floor to measure displacement. It packages these tracking metrics into a lightweight numerical datastream and transmits raw planar velocities directly over a UART serial channel to the Navigator.

The Navigator board intercepts these data streams (I2C and UART) and passes them down the internal 40-pin GPIO header into the Pi's Linux operating system for state estimation.

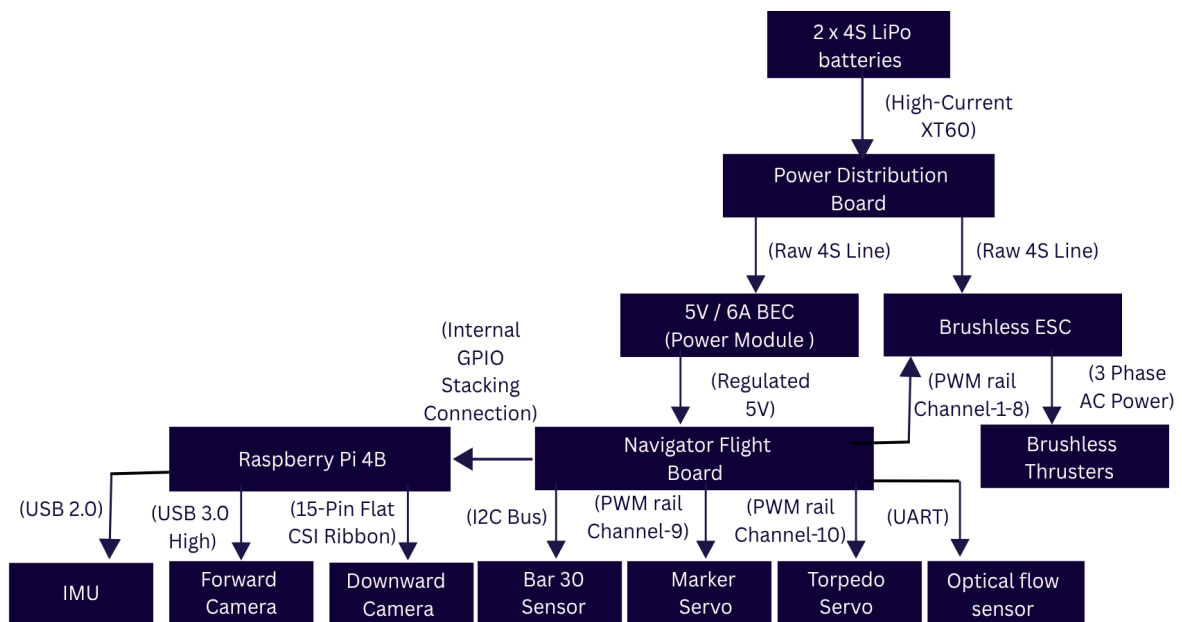
4. Closed-Loop Actuation and Payload Control

Once the ROS 2 localization algorithms on the Raspberry Pi 4B compute a spatial steering correction, the execution loop is closed via low-level signal paths as follows:

Command Transmission: The Pi flashes navigation updates back up across the Internal GPIO Stacking Connection to the Navigator's microcontroller.

Propulsion Output: The Navigator instantly translates these commands into low-level PWM signal pulses across Channels 1-8. These pulses travel along the PWM rail directly to the Brushless ESCs, dictating the exact RPM required for each Brushless Thruster to correct the vehicle's trajectory.

Payload Output: Simultaneously, mechanical payload commands are dispatched from the Navigator via dedicated servo lines:



- PWM Rail Channel-9 interfaces with the Marker Servo for precise marker dropping tasks. PWM Rail Channel-10 connects to the Torpedo Servo to actuate the torpedo reclamation mechanism.

6.5 Connection Diagram

The engineering rationale for each interface connection is:

- **USB 3.0 for Forward Camera (Intel RealSense):** Provides the high-bandwidth pipeline 5Gbps necessary to stream uncompressed color and depth frames simultaneously without introducing visual latency or frame drops.
- **15-Pin CSI Ribbon for Downward Camera:** Bypasses the USB bus to route raw video frames directly to the Pi's internal Image Signal Processor, ensuring zero-copy frame access for high-frequency OpenCV line tracking.
- **USB 2.0 Serial for IMU (XSENS):** Offers an isolated, collision-free data path to deliver high-rate 400Hz inertial telemetry reliably to the Extended Kalman Filter prediction engine.
- **Internal GPIO Stacking (Pi 4B -> Navigator):** Establishes a direct, high-density hardware connection that eliminates external signal wires,

minimizing EMI vulnerability and latency between ROS 2 nodes and I/O registers.

- **I2C Bus for Bar30 Depth Sensor:** Uses a standard, low-overhead two-wire protocol (SDA/SCL) perfectly suited for aggregating low-frequency 50 Hz hydrostatic and power telemetry on a single internal bus.
- **UART Serial Link for Optical Flow (PMW3901):** Delivers pre-calculated velocity data streams at 121 Hz over an asynchronous serial connection, preventing high-frequency coordinate vectors from flooding the shared I2C bus.
- **PWM Channels (1–10) for Servos and ESCs:** Employs hardware-timed pulse width modulation to ensure microsecond-accurate actuator control signals, isolating motor and latch timing from operating system lag.
- **3-Phase AC Output from Brushless ESCs:** Synchronizes electronic commutation by converting raw DC voltage into high-torque, reversible 3-phase AC fields required for active multi-axis subsea maneuvering.
- **Isolated Power Distribution (XT60 -> PDB -> BEC):** Employs a dedicated 5V/6A regulator to create a strict electrical barrier, filtering out inductive voltage spikes and back-EMF from the thrusters to safeguard the processing logic from brownouts.

7. Power System

7.1 Power Requirements

Component	Input Source	Voltage (V)	Current (A)	Current limit	Power (P)
Thrusters x 8	Raw Battery Power	16	6.25 x 8	25 A (Each)	800
Marker Dropper	AUX 5V/6V Rail (UBEC)	5	2	3A	10
Torpedo Dropper	AUX 5V/6V Rail (UBEC)	6	3	5A	18
Raspberry Pi	Shared Pi Power Plane	5	3	5A	15
Navigator	5V/6A BEC Module	5	0.5	6A	2.5
Forward Camera	Pi 4B USB 3.0 Slot	5	0.7	700 mA	3.5
Downward Camera	Pi 4B CSI Ribbon Slot	3.3 (Stepped	0.25	350 mA	0.82

		down by Pi internally)			
--	--	------------------------	--	--	--

7.2 Main Battery Selection

A 4S **Lithium Polymer (LiPo) battery configuration** (14.8V nominal, 16.8V fully charged) is being used in our AUV for the following reasons-

- **Voltage Matching for Thrusters:** Operating at nominal 14.8 V directly matches the optimal efficiency curve of the T200 thrusters, allowing them to deliver maximum thrust (>3.5kgf each) without exceeding thermal limits.
- **Current Draw and Heat Reduction:** Operating at 14.8 V nominal rather than a lower rail significantly reduces the total amperage required to deliver equivalent overall power. Lowering the current draw reduces resistive heating inside the sealed hull and prevents ESC thermal throttling.
- **Stable Headroom for Logic Regulators:** Maintain a sufficient voltage overhead above the 5V BEC buck-converter threshold even at low discharge levels (approx 13.2V), preventing thruster voltage sags from browning out the Raspberry Pi.
- **Energy Density vs. Mass Budget:** High C-rating LiPo chemistry delivers aggressive burst currents for fast 6-DOF maneuvers while keeping total battery mass low enough to maintain neutral buoyancy.

System Integration & Mission Parameters

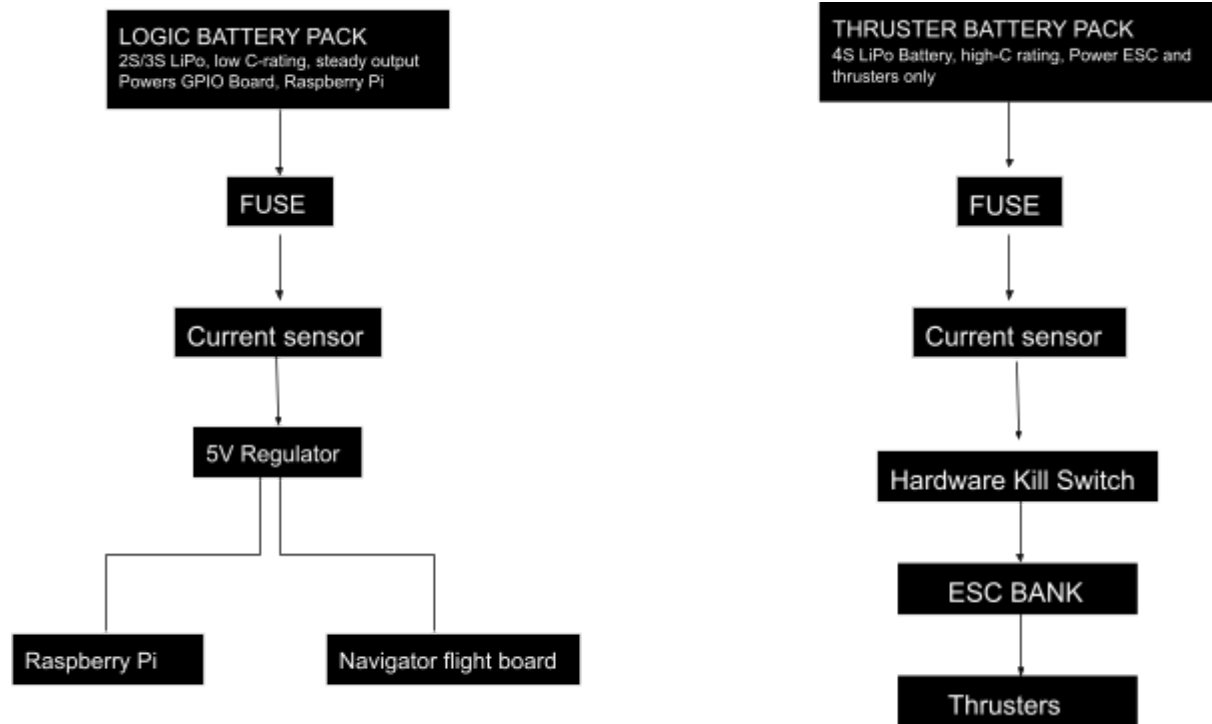
- **Dual-Pack Architecture:** Isolates high-current thruster draw from sensitive logic lines using two dedicated 4S packs (12000mAh total), suppressing EMI and voltage spikes.
- **Mission Endurance:** Under a typical continuous multi-axis current draw of 35A, the 12000mAh capacity provides up to 20.5 mins of operational runtime which is sufficient power margin to complete all mission tasks within time limit.
- **Safety & Field Maintenance:** Enclosed in a dedicated, isolated lower pod to facilitate thermal venting and allow rapid poolside pack swaps without opening the main electronics chamber.

7.3 Battery Specifications

Parameter	Specifications
Chemistry	Lithium Polymer (LiPo)
Configuration	4S
Nominal Voltage	14.8 V
Capacity	12000mAh

C-Rating	One with high C(15-20C), other with low C
Weight	5 kg
Endurance	20.5 mins

7.4 Power Distribution



7.5 Battery Management

Both battery packs (logic and thruster) are managed independently, consistent with the dual-battery architecture in Section 7.4.

1. Charging:

- Both packs are charged using a balance charger, which charges and monitors each cell individually to prevent inter-cell voltage divergence — a major cause of premature LiPo failure.
- Charging is always performed inside a LiPo-safe fire-containment bag, at a rate not exceeding 1C, and never left unattended.
- Packs are charged to full (4.2 V/cell) only shortly before a test or competition run — not stored at full charge.

2. In-mission monitoring:

- Each pack's voltage and current are monitored in real time via the INA219 sensors on the I2C bus (Section 6/8).
- Because the two packs are monitored independently, a low-voltage event on either pack (logic or thruster) is sufficient to trigger the failsafe — losing either one is mission-ending regardless of the other pack's state.

3. Storage and handling:

- Packs not in use for more than 48 hours are discharged/charged to storage voltage (~3.8 V/cell) to maximize cell lifespan.
- Each pack is visually inspected before every session for swelling, punctures, or connector damage; any damaged cell is retired, not charged.

4. Failure response:

- If a cell is found swollen or damaged, it is isolated, placed in a fire-safe container, and disposed of per LiPo safety guidelines — not charged or used again.
- Installed a kill switch to kill the power when there is some abnormality noticed in regulators collapse or batteries in order to prevent any outbursts.

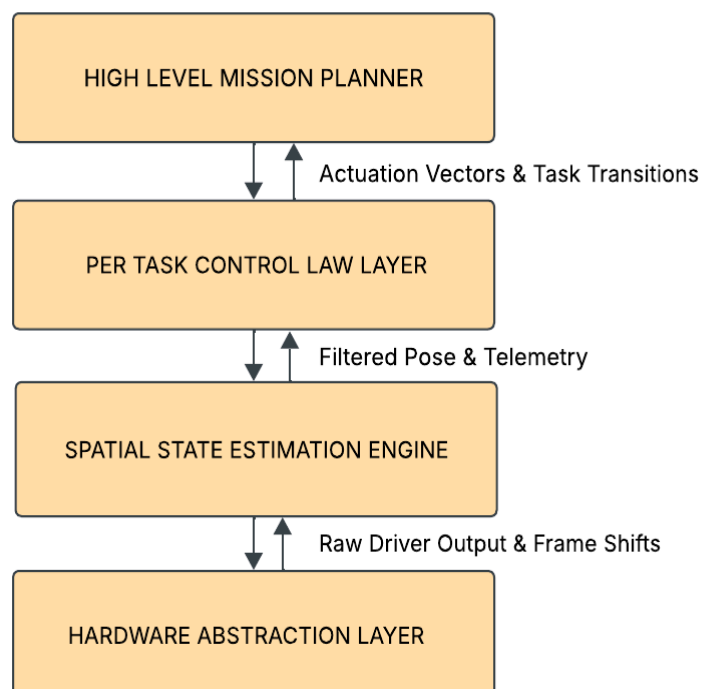
8. Software & Simulation Architecture

8.1. Overview

The vehicle's software is built completely on ROS 2 (Humble Hawksbill) running on Ubuntu 22.04 LTS linux kernel patched with the real time framework. The foundation ensures strict deterministic execution of critical navigation nodes and minimises latency during high frequency sensor processing.

8.2. Architectural Details

The architecture follows a decoupled, four layered layout to isolate systems or subsystem dependencies.



1. Hardware Abstraction Layer (HAL) :

The lowest layer serves as the software bridge between physical peripheral buses and the ROS 2 computing graph, executing via dedicated C++/Python nodes on the Raspberry Pi 4B and the Robotics Navigator board:

- **Sensor I/O Bridges** : Drives high frequency communication protocols to serialize the incoming payloads, managing I2C lines for the Bar30 pressure sensor, SPI/USB streams for the XSENS 630R MTi IMU, and serial UART packets from the PMW3901 optical flow breakout board.
- **Actuator Dispatcher** : Subscribes to unified force allocation arrays and translates them into raw hardware level timed register changes on the Navigator's multi channel PWM chip. These drive the 6 ESCs and the mechanical payload deployment servos.
- **Power Plane Monitoring** : Polls the dual INA219 power monitors over I2C at regular intervals to broadcast the electrical health, voltage drop, and current consumption profile of the split logic and thruster battery rails.

2. Spatial State Estimation Engine (Visual-Inertial EKF) :

Operating underwater completely eliminates access to GPS and renders standard magnetometers unreliable near pool walls or metallic arena frameworks. To establish a robust, shared infrastructure for navigation, a centralized Extended Kalman Filter (EKF) node is deployed using the `robot_localization` package to track state transitions across a local coordinate frame:

- **Prediction Phase**: Installs high-frequency 6-DOF inertial datasets (linear accelerations and angular velocities) from the XSENS IMU to drive the state prediction equations.
- **Correction Phase (Planar Odometry)**: a downward facing PMW3901 laser optical flow sensor tracks frame to frame pixel displacements along the pool floor. By integrating these spatial velocities and dynamically scaling them

using the absolute depth from the pressure sensor, it computes accurate metric linear velocities (v_x, v_y) to bound the IMU's raw integration drift.

- **Z-Axis & Boundary Correction:** Provides drift free vertical axis updates via absolute hydrostatic metrics from the Bar30 sensor ($P=\rho gh$). The EKF uses these parameters to publish a unified, smooth localization state topic (/odometry/filtered) consumed by all tracking loops.

3. OpenCV Perception Layer:

To maximize processing throughout and minimize the computational load on the Raspberry Pi 4B's CPU, the system utilizes an optimized OpenCV C++ pipeline focused on structural features rather than intensive deep-learning tracking networks:

- **Lightweight Color Segmentation:** Targets highly saturated, distinct tournament colors (e.g., orange path markers, buoy surfaces). The perception node transforms raw frames into HSV space, applies fine-tuned threshold bounding masks, and extracts the target spatial centroid at full camera frames with zero training data overhead.
- **Geometric Feature Extraction:** When color contrast degrades (e.g., bare grey PVC pipes on the lane gate), the node executes a bilateral blur filter combined with a Canny edge detector and a Hough Line Transform to mathematically isolate intersections and guide alignment tasks.

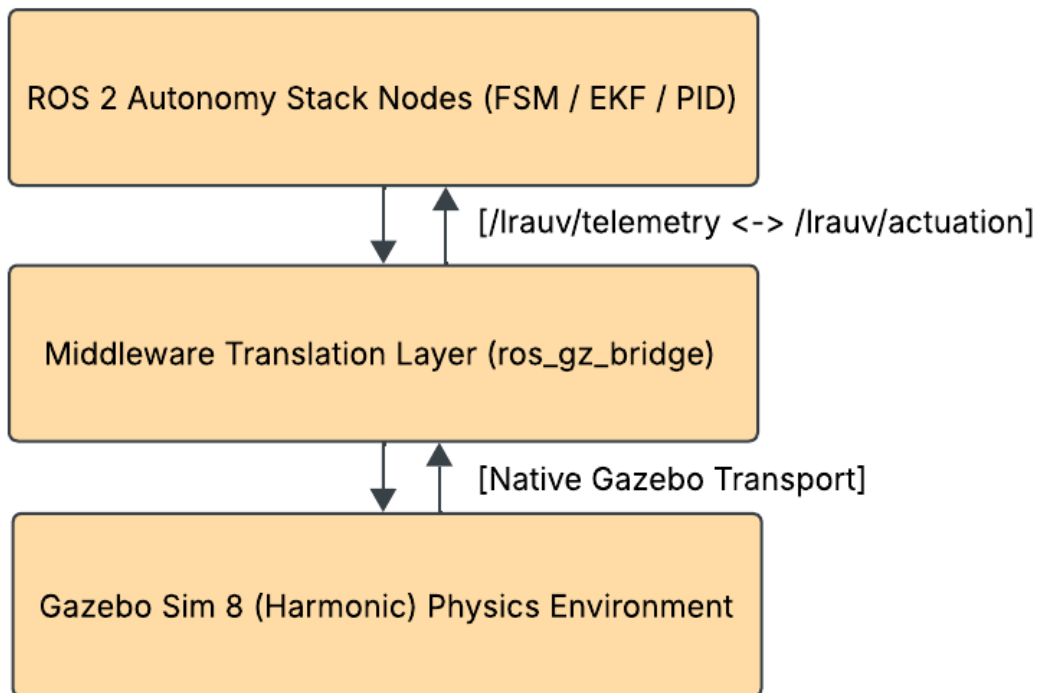
4. High-Level Mission Planner (Finite State Machine) :

The macro execution of the mission run is orchestrated by a high-level Finite State Machine (FSM) node. The FSM manages sequential task routing and updates setpoints based on target completion flags or strict regional node timeouts:

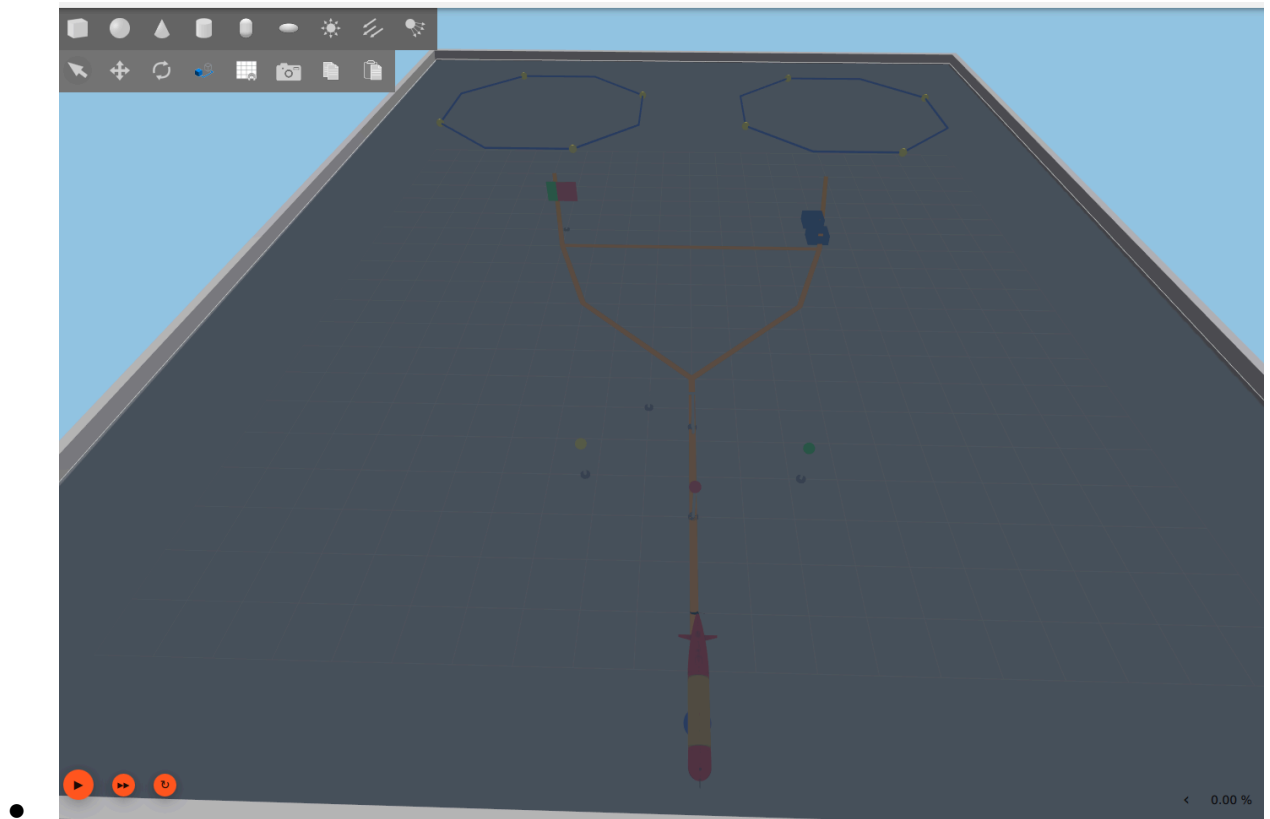
- **Dynamic Time-Budget Assessment:** Before tackling split routes, the FSM actively cross-references the vehicle's remaining operating runtime against the tournament's 30-minute competition limit. If the time budget is constrained, it modifies the graph, dropping non-essential tasks to navigate safely to the final recovery octagon.
- **Parallel Safety Kernel:** Runs continuously alongside the autonomy code to poll health metrics. If an emergency threshold is breached (e.g., INA219 battery voltage collapse, moisture flags, or internal thermal limits inside the acrylic capsule), the kernel executes a hard system abort—zeroing thruster output, dropping payload weights, and exploiting the vehicle's inherent positive buoyancy ($B>W$) to initiate a passive emergency ascent.

8.3 Simulation Environment (SITL)

To enable rigorous verification of autonomy algorithms and control gains without risking physical components, a high fidelity Software in the Loop (SITL) testbench is built directly on Gazebo Sim 8 (Harmonic):



- **World Generation:** The arena world file (`save_arena.sdf`) is generated programmatically from a custom script, ensuring that all target elements (buoys, PVC lines, bins, octagons) accurately match the competition dimensions rather than being estimated manually.
- **Hydrodynamics & Physics Configuration:** Runs the Open Dynamics Engine (ODE) configured with a strict 1 ms fixed step size. A global Gazebo fluid buoyancy plugin enforces uniform fluid density ($\rho=1000 \text{ kg/m}^3$), automatically generating localized buoyancy forces by checking the displaced water volume of each vehicle collision link.
- **Middleware Interfacing:** Maps data via the `ros_gz_bridge` using a declarative configuration file (`save_bridge.yaml`). Explicit directionality rules are enforced (`GZ_TO_ROS` for sensor telemetry, `ROS_TO_GZ` for thruster thrust arrays), fully preventing feedback discovery loops and isolating runtime tasks on a dedicated `ROS_DOMAIN_ID`.



Simulation Visuals

8.4. Digital Twin Vehicle Profile

The virtual environment completely avoids the use of underactuated, single propeller torpedo templates. Instead, it deploys a custom holonomic 6 thruster digital twin that mirrors the actual physical parameters, mass limits, and thruster placement of our physical vehicle framework:

1. Actuator Grid Layout

The vehicle simulation exposes 6 independent motor joint interfaces matching the actual hardware thruster arrangement:

1. **Horizontal Propellers (T1–T4):** Four thrusters are arranged at $\pm 45^\circ$ angles in the horizontal plane, placed symmetrically around the geometric center of gravity. This layout provides full, coupled control vectors over three degrees of freedom: Surge (F_x), Sway (F_y), and Yaw (M_z).
2. **Vertical Propellers (T5–T6):** Two thrusters are mounted vertically near the longitudinal center of mass. They are driven identically to handle absolute

Heave (Fz) adjustments while completely canceling out dynamic pitching moments.

2. Virtual Sensor Plugins

The virtual twin asset is configured with low-level sensor plugins to accurately mimic the real world sensor data rates, noise characteristics, and interfaces:

- **Virtual IMU Plugin:** Replicates the data structure of the XSENS 630R MTi, injecting realistic high-frequency white noise and bias instabilities into linear acceleration and angular velocity frames.
- **Hydrostatic Pressure Plugin:** Simulates the Bar30 sensor, reading the absolute depth coordinate (Z) within the Gazebo engine and converting it to an ambient pressure output stream.
- **Optical Flow Emulation:** Replicates the PMW3901 breakout board by calculating the vehicle's relative tracking vectors over the pool floor tile textures, exporting data over a simulated serial interface.
- **Camera Array Plugins:** Spawns a forward facing RGB camera plugin mimicking the Intel RealSense D435i lens profile, along with a wide angle downward facing camera tracking the pool floor path. This provides the exact image streams needed to test the perception algorithms.

9. Navigation Stack

9.1 Sensor Fusion and Output Data Packets

QoS Profile Rationale:

- Sensor streams (/imu/data, /opticalsensor/data /depth, /camera/color/image_raw, /downward/image_raw , /thrust_data or /motor_rpm):
Best Effort delivery is used since occasional packet loss is acceptable and minimising latency takes priority over guaranteed delivery. A shallow queue depth ensures that stale sensor readings or blurred visual frames are instantly overwritten, preventing old data from accumulating and introducing processing lag between update cycles.
- Thrust commands (/thrust_pwm_cmd):
Reliable with Depth 1 ensures every command either reaches the ESC node or triggers a retransmit. A depth of 1 ensures the ESC always acts on the most recent command without queuing outdated ones.
- EKF state (/ekf/state):

Reliable with Depth 1 is used since both the control and planning nodes require the freshest consistent state estimate and must not miss an update cycle.

- Waypoints (/mission/waypoints):

Reliable with Transient Local durability ensures that the control_node receives the waypoint path even if it comes online after the planning node has already published, avoiding a missed-message race condition at startup.

9.2 Frame Transformation

Each sensor or component operates with different frames of reference.

- **IMU**

It operates with the 0 heading aligned to the magnetic poles. This requires a frame transformation to the geographic poles and then to the ECEF and NED frames.

- **Pressure sensor**

It operates in the vessel's local frame. This is a moving frame of reference. It is used in the local frame and converted to a global frame for mapping, evasion and detection.

- **Forward camera (Intel RealSense)**

Operates in its own camera optical frame (X right, Y down, Z out of the lens), which is a moving frame rigidly fixed to the vehicle. A static extrinsic transformation (from calibration) rotates and translates it into the vehicle's local frame, and converted to the global (ECEF $\rightarrow$ ENU) frame using the vehicle's current pose. Used for obstacle detection, object recognition, and mapping.

- **Downward camera (CSI Path Tracker)**

It operates in a vertically pitched optical frame (Z-axis projecting straight down toward the pool floor). This is a floor-facing moving reference frame. For line/path following it is used directly in the local (body) frame, since tracking only needs relative offset and heading error. When those detections must be mapped, the extrinsic transform takes it to the body frame and then to the global/ENU frame.

- **Optical flow sensor**

Operates in its own Local Sensor Frame follows the same standard convention as forward camera. It undergoes rotational and spatial scale transformations into the Vehicle Body Frame so the EKF can interpret the data.

9.3 Extended Kalman Filter based Localization

A 12-state Extended Kalman Filter (EKF) is implemented via the ROS 2 robot_localization package for 3D state estimation. The planar coordinate system is expanded to the standard ENU (East-North-Up) convention. The state vector is:

$$\mu = [x, y, z, \Phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \omega, \alpha, \beta]^T$$

Where:

- (x, y, z) is the 3D position of the robot in the local ENU frame.
- (Φ, θ, ψ) are the Euler angles for roll, pitch, and heading (yaw).
- $(\dot{x}, \dot{y}, \dot{z})$ are the linear velocities in the respective axes.
- (ω, α, β) are the angular velocities.

The covariance matrix P is a 12 x 12 matrix with all non-diagonal values initialized to 0:

$$\sigma = \text{diag}[\sigma^2 x, \sigma^2 y, \sigma^2 z, \sigma^2 \Phi, \sigma^2 \theta, \sigma^2 \psi, \sigma^2 \dot{x}, \sigma^2 \dot{y}, \sigma^2 \dot{z}, \sigma^2 \omega, \sigma^2 \alpha, \sigma^2 \beta]$$

The Linearized Jacobians (G, V, H) are derived analytically and applied at the high-frequency update rate of the XSENS IMU i.e 400Hz.

1. Prediction Step

The nonlinear process model propagates the state forward using bias-corrected IMU readings at each timestep Δt :

$$\begin{aligned} x_{+1} &= x_{\square} + \dot{x}_{\square} \Delta t \\ y_{+1} &= y_{\square} + \dot{y}_{\square} \Delta t \\ \psi_{+1} &= \psi_{\square} + \omega_{\square} \Delta t \\ \dot{x}_{+1} &= \dot{x}_{\square} + (a_x^M - b_{ax}) \cos \psi_{\square} \Delta t \\ \dot{y}_{+1} &= \dot{y}_{\square} + (a_y^M - b_{ay}) \sin \psi_{\square} \Delta t \\ \omega_{+1} &= \omega_{\square} + (\omega^M - b_{\omega}) \Delta t \end{aligned}$$

$b_{ax}, b_{ay}, b_{\omega} = \text{constant}$ (random walk model),

$\phi^M = \text{analytical value from motion model of the system.}$

a^M represents the analytical acceleration value derived from the IMU, mapped into the ENU frame

The linearized Jacobian G (12x12) is evaluated at each prediction step for covariance propagation:

$$P_{+1}^- = G_{\square} P_{\square} G_{\square}^T + V_{\square} Q V_{\square}^T$$

where Q is the process noise covariance (tuned empirically) and V is the noise input Jacobian.

2. Correction Step

The measurement model relies on the Bar30 depth sensor (50Hz) and the OpenCV visual pipeline. When a depth reading or visual pose target arrives, the measurement model isolates the observable states:

$$z_k = [x_{vis}, y_{vis}, z_{bar30}, \Phi_{vis}]^T = H X_k + v_k$$

$$H = \begin{bmatrix} 1, & 0, & 0, & 0, & 0, & 0, & 0, & 0, & 0 \\ 0, & 1, & 0, & 0, & 0, & 0, & 0, & 0, & 0 \end{bmatrix} \quad (2 \times 9 \text{ matrix})$$

The Kalman gain and state update are then computed as:

$$K_k = P^-_k H^T (H P^-_k H^T + R)^{-1}$$

$$X_k = X^-_k + K_k (z_k - H X^-_k)$$

$$P_k = (I - K_k H) P^-_k$$

9.4 Path Planning and Waypoint Generation

Mission waypoints are generated using a minimum-jerk trajectory, derived from a 6th-order Euler-Lagrange formulation that yields a 5th-degree polynomial in each axis (x, y, z). Motion constraints — maximum and minimum surge speed, turning radius, climb/dive rate, and angular velocity limits — are applied to solve for the polynomial coefficients, producing a smooth, dynamically feasible path that the vehicle can physically follow without abrupt acceleration changes

The resulting trajectory is sampled to produce intermediate waypoints between the current EKF-estimated pose and the next task goal

Waypoints are published by the path_planning node on /mission/waypoints as a nav_msgs/Path message and consumed by the control node, which computes cross-track and heading error to execute the path in real time.

Waypoint Representation:

Each waypoint in the path is defined by the following fields:

Field	Value	Description
seq	0,1,2 ...	Waypoint index; 0 is the launch/home position
x	metres	East position in the local ENU frame
y	metres	North position in the local ENU frame

z	metres	Depth below surface(from the pressure sensor)
yaw	radians	Desired heading at the waypoint(NaN = maintain course)
task	enum	Mission action at waypoint (TRANSIT, TOUCH_BUOY, DROP_MARKER, FIRE_TORPEDO, SURFACE)
accept_radius	metres	Waypoint is considered reached within this radius
hold_time	seconds	Time to hold position at waypoint before continuing

Example Mission Sequence

A representative waypoint set following the NIOT mission timeline (launch → dive → path → buoy → gate → fork → octagon):

seq	x	y	z	yaw	task	radius	hold
0	0.00	0.00	0.00	0.00	TRANSIT	0.30	0
1	0.00	0.00	1.20	0.00	TRANSIT	0.20	2
2	3.50	1.00	1.20	0.30	TRANSIT	0.30	0
3	5.00	2.20	1.20	0.30	TOUCH_B UOY	0.25	3
4	8.00	3.00	1.20	0.00	TRANSIT	0.30	0
5	11.00	4.50	1.50	0.00	DROP_MA RKER	0.25	3
6	14.00	6.00	0.00	0.00	SURFACE	0.40	0

Execution Procedure

The planned path is executed entirely within the ROS 2 stack (no external autopilot upload):

```
# path_planning node computes the minimum-jerk  
trajectory and publishes waypoints  
ros2 topic echo /mission/waypoints  
# verify the published path
```

```
# control_node subscribes to /mission/waypoints and /ekf/state,  
# computes cross-track + heading error, and outputs thrust commands  
ros2 topic echo /thrust_pwm_cmd  
# confirm control output  
# mission is armed and sequenced by the state machine  
ros2 service call /mission/start  
std_srvs/srv/Trigger
```

The control_node advances through the sequence as each waypoint's acceptance radius is met, holding for the specified hold_time at task waypoints (e.g., pausing to touch a buoy

or fire a torpedo). Because planning and control run natively in ROS 2 against the EKF state estimate, the vehicle maintains full autonomy without relying on a GPS-referenced autopilot layer.

10. Control System Design & Tuning

10.1. Cascaded PID Architecture

The vehicle utilizes a cascaded, task-switched PID control framework. An outer supervisory layer selects the active mission task. Each task then activates one or more dedicated inner loop PID controllers that consume the Extended Kalman Filter state (/odometry/filtered) or visual pixel error inputs to output spatial thrust demands.

The vehicle actively regulates four independent degrees of freedom : Surge, Sway, Heave, and Yaw. Roll is passively stabilized by the mechanical hull architecture, which structural elements keep naturally righted by keeping the center of buoyancy well above the center of gravity. Pitch is kept minimized by matching commands symmetrically across the two vertical thrusters. Multiple PID loops run simultaneously during multi-axis maneuvers.

Every loop implements the standard discrete PID formulation running natively within its ROS 2 controller node:

$$e_k = \text{setpoint}_k - \text{measured}_k$$

$$\text{integral}_k = \text{integral}_{k-1} + e_k \cdot \Delta t$$

$$\text{derivative}_k = -(\text{measured}_k - \text{measured}_{k-1}) / \Delta t$$

$$u_k = K_p \cdot e_k + K_i \cdot \text{integral}_k - K_d \cdot \text{derivative}_k$$

1. Anti-Windup Protection

Integral accumulation is clamped whenever a thruster's allocated command saturates, preventing severe tracking overshoot when the vehicle commands maximum current limits on an axis.

2. Derivative-on-Measurement

The derivative term acts directly on the EKF state or perception measurement rather than the raw tracking error. This mathematically avoids the "derivative kick" (sudden spikes in actuator commands) that would otherwise occur whenever a new target setpoint coordinate is injected into the controller.

3. Image Based Visual Servoing (IBVS)

For proximity maneuvers (Buoy Touch, Torpedo Alignment), the loops are structurally identical PID nodes, except the error measurement is parsed as a pixel-space offset or bounding box surface area from the camera framework rather than a metric EKF state coordinate.

10.2. 6-Thruster Allocation Matrix

The PID control nodes output a desired global force/moment vector, or wrench (τ), along the vehicle's structural axes:

$$\tau = [F_x, F_y, F_z, M_z]^T$$

Where F_x is surge force, F_y is sway force, F_z is heave force, and M_z is yaw moment. Because the vehicle has 4 controllable degrees of freedom but utilizes 6 physical thrusters, this vector must be split across the physical hardware actuators using a specialized configuration matrix:

$$T = B^+ \cdot \tau$$

Where $T = [T_1, T_2, T_3, T_4, T_5, T_6]^T$ represents the individual thruster forces in Newtons, B is the 4×6 structural geometry matrix derived from each thruster's exact

cross-chassis mounting coordinate and thrust unit vector, and B^+ is its Moore Penrose pseudo-inverse.

This formulation distributes the workload evenly across the horizontal thruster cluster (T1 - T4), managing translation vectors seamlessly. The vertical cluster (T5 - T6) receives equal allocation inputs to generate clean heave transitions without introducing dynamic pitch torque.

10.3. Nonlinear PWM Conversion

The calculated individual forces T_i (in Newtons) must be converted into low-level microsecond PWM pulses before transmission to the Navigator's ESC channels. Because the Blue Robotics T200 thrusters exhibit a highly nonlinear force output profile characterized by a small deadband near the neutral point (1500 μ s) and asymptotic thrust saturation near the operational edges a linear map is insufficient.

The stack utilizes a custom look-up table combined with a fitted 5th-order polynomial equation to capture this nonlinear thruster mapping profile:

$$PWM_i=f(T_i)$$

This model applies strict saturation bounding limits to ensure individual commands never exceed safe hardware limits, communicating feedback flags directly to the anti-windup algorithms to decouple and preserve control loop integrity.

10.4. Controller Loop Tuning Parameters

The initial gains are tuned inside the Gazebo Software-in-the-Loop (SITL) platform against simulated hydrodynamic drag coefficients, establishing standard values prior to pool validation.

Controller Axis	Kp	Ki	Kd	Target Bandwidth	Technical Purpose
Heave / Depth (Z)	4.0	0.15	1.2	~1.0 Hz	Driven by the Bar30 sensor; dominates depth-dives, gate pass-over holds, and surfacing maneuvers.
Yaw / Heading (ψ)	2.2	0.04	0.7	~0.8 Hz	Common steering kernel shared across path tracking and target alignment tasks.

Surge (Path Cruise)	12.0	0.30	1.5	~1.0 Hz	Stabilizes steady forward velocity vectors while yaw handles line tracking error adjustments.
Surge/Sway (Station)	6.0	0.20	1.0	~0.6 Hz	Slower, high-precision loop for bin drop positioning; keeps overshoot strictly bounded.
IBVS Yaw (Visual Lock)	3.0	0.05	0.9	~1.2 Hz	Tracks pixel centroid displacement errors; smooths frame jitter via an EMA pre-filter.
IBVS Heave (Visual Lift)	3.5	0.10	1.0	~1.0 Hz	Corrects vertical error offsets relative to known target geometries.

Tuning & Validation Pipeline

Tuning adheres to a strict bottom-up validation methodology to maximize safety and efficiency:

1. **Perception Isolation:** Autonomy tracking filters are run against simulated visual data inside Gazebo, comparing calculated vision tracking errors against the simulator's true coordinate frames to verify tracking precision.
2. **Isolated Task Tuning:** PID parameters are tuned for each individual control loop independently by injecting step-response targets into isolated simulation runs.
3. **FSM Integration:** The complete, tuned control nodes are chained together inside the overarching state machine sequencer only after passing individual axis validation. Loops that share a physical control axis utilize a single unified gain profile to maintain predictable vehicle dynamics across mission task transitions.

11. Communication systems

11.1 System Overview

Link	Bus/interface	Purpose
------	---------------	---------

Raspberry Pi 4B – Navigator	40-pin GPIO header/Internal serial	Pass sensor data up, send steering commands down
Intel RealSense D435i –Pi4B	USB 3.0	Forward colour/depth frames for target lock (buoy, torpedo cutout)
Downward Pi Camera Module 3-Pi 4B	15-pin CSI ribbon	Pool-floor frames for orange-path tracking
Bar30 (MS5837) – Navigation	I2C(50hz)	Depth / pressure and water temperature
INA219(2) – Navigator	I2C	Per-pack voltage and current monitoring
XSENS MTi-630R – Pi 4B	USB /serial(400hz)	6-DOF inertial data + magnetometer
Navigator – ESCs (6 T200)	PWM	Thruster speed commands
Navigator – DSServo DS3225/DS3235	PWM	Marker drop and torpedo release actuation

11.2 Kill Switch Compliance

A magnetically actuated switch on the hull exterior — reachable by a diver or judge without opening any enclosure — breaks the main power line of the thruster battery pack, removing power from all six T200 ESCs and stopping propulsion within one control cycle.

A software disarm backs up the hardware cut. The Navigator's ARM/DISARM state is exposed to the ROS 2 mission node; any on-board abort trigger — leak detection, either battery pack under-voltage (INA219), or a Mission Sequencer (FSM) task

timeout — commands zero throttle to the ESCs and halts all setpoint publishing on /thrust_pwm_cmd.

12. Safety Analysis

12.1 Safety Features

- Hardware kill switch: external magnetic switch cuts thruster-pack power to all 6 ESCs; separate logic pack keeps the Pi and logging alive.
- Software failsafe: on-board abort on leak, either-pack under-voltage (INA219), or FSM task timeout -> ESCs to zero throttle.
- Shrouded propellers: guards on all 6 T200 thrusters (mandatory inspection item).
- Separated battery: dual 4S LiPo in a vented enclosure, isolated from the electronics hull.
- Buoyancy reserve: trimmed slightly positive -> vehicle floats to the surface on any power or software failure.
- Leak protection: acrylic hull with O-ring end caps; vacuum-tested before every session, plus in-hull leak indication.
- LiPo handling: balance-charged at $\leq 1C$ in a fire-containment bag; kept at storage voltage when idle.
- Software limits: thruster-saturation clamping, PID anti-windup, and hard FSM task timeouts.

12.2 Hazard Analysis

A Failure Modes and Effects Analysis (FMEA) is used to consider each credible failure, its effect, and the mitigation already in place. The entries below are representative.

Failure mode	Effect	Severity	Mitigation
Water ingress into electronics hull	Short circuit; loss of vehicle	High	Vacuum test each session; O-ring end caps; leak indication → abort & surface
Single thruster / ESC failure	Reduced control authority; veer	Medium	Redundant 6-thruster vectored layout; allocation redistributes wrench; fuse per pack
LiPo cell fault (swelling/short)	Fire / gas in battery hull	High	Separate vented battery enclosure; INA219 monitoring; kill switch; fire-bag charging

Control software hang	No commands; uncontrolled drift	Medium	Hardware kill switch cuts propulsion directly; positive buoyancy floats vehicle up
EKF divergence (sensor dropout)	Bad position estimate; off-course	Medium	Multi-sensor fusion (IMU+DVL+depth+visual landmarks); FSM task timeout aborts
Logic battery under-voltage	Loss of compute mid-mission	Medium	Per-pack low-voltage failsafe; either pack low ends mission safely

13. Simulation and Testing

The software stack is developed and validated in simulation before any in-water trial, so that logic errors are found at zero risk to the hardware. The simulator is built on Gazebo Harmonic with ROS 2 Humble, bridged by a Harmonic-compatible build (ros-humble-ros-gz-harmonic) that translates Gazebo transport topics to ROS 2 topics through a declarative YAML configuration, with direction (GZ_TO_ROS for telemetry, ROS_TO_GZ for actuation) explicitly defined to prevent feedback loops.

- Arena — the world is generated programmatically to match the competition tank exactly (25 m × 20 m, 2–2.5 m water depth), using an ODE physics step and a uniform fluid-buoyancy plugin (1000 kg/m³) that applies Archimedes' principle to all submerged bodies.
- The vehicle model — the my_auv platform (derived from the MBARI Tethys architecture) provides simulated hydrodynamics. A modelled Doppler Velocity Log (DVL) supplies velocity aiding to bound dead-reckoning drift, standing in for the underwater X/Y position reference that GPS cannot provide.

13.1 Simulation Test Results

Behaviour	Metric	Status	Pass Criterion
EKF convergence from cold start	State - estimate error after 30s	pass	<1 m
Depth hold(Bar30/EKF z)	Steady - state error	pass	< ± 5 cm
Path following (downward cam)	Lateral offset RMS	pass	≤ 8 cm

Buoy touch	Centroid convergence time	pass	≤ 3 s
Gate pass-over	Heading hold error	pass	$\leq \pm 4^\circ$
Station keeping / hover hold(60s)	Position drift	pass	<0.3 m
Bin drop (multi-axis PID)	Position variance at trigger	pass	Below drop threshold
Torpedo alignment	Cutout centroid lockout time	pass	Centred within tolerance < N frames
Kill-switch response	Time for all 6 thrusters to zero	pass	< 200 ms
Surface in octagon	Lateral drift during ascent	pass	≤ 20 cm

13.2 Hardware- in - the - loop results

HITL testing runs the real on-board electronics (Raspberry Pi 4B, Navigator, XSENS MTi-630R, Bar30, ESCs) on the bench with the thrusters spun in air, to verify integration that pure simulation cannot — correct wiring, driver behaviour, message timing, and actuator response. The planned bench tests and their pass criteria are listed below.

Test	Location	Date (Planned)	Pass Criteria
Sensor data flow	IIT KGP Lab (bench)	Week N-1	IMU (400 Hz), Bar30 (50 Hz) and both cameras publish valid ROS 2 topics
ESC / thruster signal (motors in air)	IIT KGP Lab (bench)	Week N-1	Correct PWM on all 6 T200 channels; spin direction correct
End-to-end latency	IIT KGP Lab (bench)	Week N-1	Perception -> EKF -> PID -> /thrust_pwm_cmd under 50 ms
Kill-switch response	IIT KGP Lab (bench)	Week N-1	All 6 thrusters stop under 200 ms; Pi stays powered and logging

Test	Location	Date (Planned)	Pass Criteria
Payload actuation	IIT KGP Lab (bench)	Week N-1	DS3225 marker drop and DS3235 torpedo release fire on command
EKF sensor-fusion sanity (hand-moved)	IIT KGP Lab (bench)	Week N-1	EKF state tracks motion; no divergence

13.3 In-Water Testing Timeline

Stage	Objective	Success criterion
1. Static ballast & leak check	Confirm neutral/slightly-positive trim and a dry hull	Holds depth; vacuum holds; no ingress
2. Manual / tethered motion	Confirm thrusters and low-level control produce stable motion	Stable surge, heave, yaw response
3. Single behaviours	Depth hold, heading hold, orange-path follow — in isolation	Each behaviour meets its desired result in section 10.1
4. Task chains	Dive → path → buoy; path → gate; fork → marker/torpedo	Consecutive tasks complete without operator input
5. Full mission rehearsal	Complete NIOT run end-to-end in a competition-like pool	Autonomous launch-to-octagon within time limit

14. Competition Timeline

Milestone	Objective	Duration	Status
Preliminary Design Report (PDR)	Lock architecture and major component choices	2 weeks	In progress
Detailed design & procurement	Finalise CAD; order parts (allow shipping lead time)	3-4 weeks	Pending
Fabrication & assembly	Build frame, hull, enclosures; integrate electronics	4 weeks	Pending

Software integration + SITL/HITL	Validate full mission in sim and on the bench	3 weeks	Pending
In-water testing	Buoyancy/leak checks → single behaviours → task chains	3 weeks	Pending
Full mission rehearsals	End-to-end autonomous runs in a competition-like pool	2 weeks	Pending
Competition	NIOT Student AUV Competition run	3-day event	Target date — insert